HANDHELD DEVICE PROGRAMMING I

(REACT NATIVE)

CHAT APPLICATION

PROJECT DOCUMENTATION

STUDENT NAME : **A.P.L Osini Navoda Liyanage**

NIC : **200380111705**

PROJECT NAME : **Orbit Chat App**

Acknowledgement

I would like to express my sincere gratitude to everyone who contributed to the successful completion of the Orbit Chat project.

- To my Project Guide/Instructor: Thank you for your invaluable guidance, technical insights, and constant encouragement throughout this journey. Your feedback was crucial in shaping the project into what it is today.
- To my Colleagues and Peers: Thank you for the brainstorming sessions, code reviews, and moral support. Your collaboration made problem-solving easier and more enjoyable.
- To the Open-Source Community: A huge thank you to the developers of React Native, Expo, Java EE, Hibernate, and all the libraries used. Your work forms the backbone of this application.
- To My Family and Friends: Thank you for your unwavering support, patience, and understanding during late nights and intense debugging sessions. Your belief in me kept me going.

This project wouldn't have been possible without the collective effort and support from all of you. Thank you!

Table of Contents

Table of Figures

Project Overview

Orbit Chat is a cross-platform, real-time messaging application developed as a comprehensive learning project. Inspired by popular messaging platforms like WhatsApp and Telegram, Orbit Chat enables users to register, authenticate, manage contacts, and exchange instant text messages with individuals in real-time.

Built using the React Native (Expo) framework for the frontend and a Java EE (Hibernate) backend with WebSocket technology, the application demonstrates full-stack development skills. It emphasizes a clean, user-friendly interface with a distinctive cosmic purple aesthetic, supporting both light and dark modes.

The primary purpose of Orbit Chat is to provide a functional and visually appealing messaging experience while showcasing the integration of modern technologies for real-time communication. The scope of this project includes fundamental chat application features such as user onboarding, persistent authentication, real-time messaging with delivery/read receipts, contact management, and a settings section for user customization. It is designed and tested for deployment on both Android and iOS mobile devices.

Objectives

The primary objectives of the Orbit Chat project are:

- To Implement a Functional Real-Time Communication System: Leverage WebSocket technology to enable instant messaging between users with minimal latency.
- To Design an Intuitive and Visually Appealing User Interface: Create a user-friendly experience with a distinctive cosmic purple theme, smooth animations, and support for both light and dark modes using React Native and Expo.
- To Develop a Secure User Authentication and Data Handling Mechanism: Implement a robust sign-up and sign-in process, manage user sessions persistently, and ensure contact and message data is handled appropriately on both the client and server sides.
- To Gain Practical Full-Stack Development Experience: Apply and integrate knowledge of frontend technologies (React Native, TypeScript, Reanimated) with a backend built using Java, Hibernate ORM, and WebSocket API.
- To Provide Essential Chat Application Features: Deliver core functionalities such as user onboarding, contact management, real-time messaging with delivery/read receipts, and a customizable user profile and settings section.

Technology Stack

- Frontend:
    - React Native (Expo): Used for building the cross-platform mobile application, allowing deployment to both Android and iOS from a single codebase. Developed using JavaScript/TypeScript.
    - UI Library/Framework: NativeWind (based on Tailwind CSS) is employed for efficient and consistent styling across the application interface.

- Backend:
    - Java EE: The server-side logic is implemented using standard Java technologies.
    - Hibernate ORM: Utilized as the Object-Relational Mapping (ORM) tool to simplify database interactions and manage persistent data (e.g., Users, Chats, Friends).
    - WebSocket API: Provides the core mechanism for establishing persistent, bidirectional communication channels between the client and server, enabling real-time messaging.
- Database:
    - MySQL: Serves as the relational database management system for storing essential application data, including user profiles, contact lists, and chat message history.
- Communication:
    - WebSocket Protocol: The primary method for real-time, bidirectional data transfer (sending/receiving messages, status updates).
    - Fetch API: Used for making standard HTTP requests, primarily for initial user authentication flows (sign-in/sign-up) if applicable alongside WebSocket.

System Architecture

The Orbit Chat application follows a client-server architecture, logically divided into four main layers to facilitate real-time messaging and data management. Below is a breakdown of each layer and its components based on the technologies used in Orbit Chat:

1. Client Layer (Mobile Application Frontend):
    - Technology: Built using the React Native (Expo) framework.
    - Role: This is the user-facing layer where users perform actions such as signing up/in, viewing their contact list, sending and receiving messages, and managing their profile and settings. It provides the complete graphical user interface (GUI) and handles user interactions. The UI is styled using NativeWind (Tailwind CSS) for a consistent and responsive design across Android and iOS devices. React Navigation manages screen transitions, and React Context API handles global state like authentication and theming.

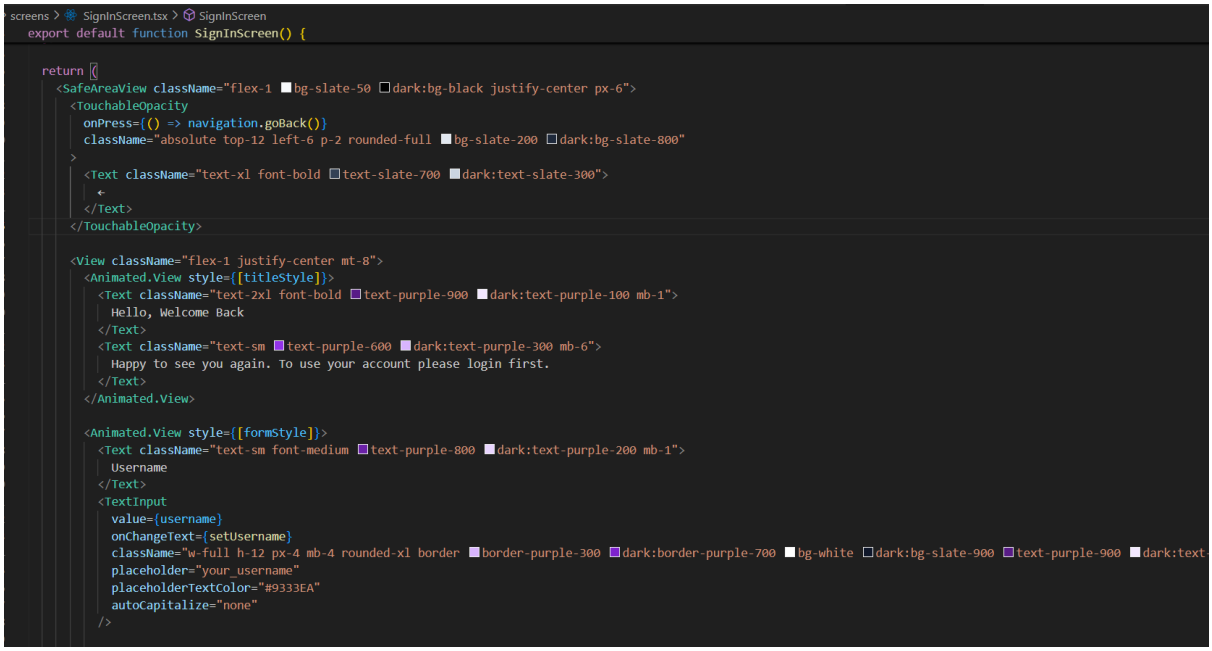


Figure 1:nativewind usage

2. Server Layer (Application Backend):
    - Technology: Developed using Java EE technologies, specifically leveraging standard Servlet API and javax.websocket.
    - Role: This layer is the core processing unit of the application. It hosts the business logic for user management, contact handling, and chat functionalities.

- Components:
    - ChatEndPoint: A @ServerEndpoint class responsible for managing persistent WebSocket connections with individual clients. It receives and dispatches real-time messages, handles user presence (login/logout), and triggers appropriate services.
    - UserService, ChatService: These service classes contain the logic for database operations related to users (registration, profile updates, friend lists) and chats (message history retrieval, sending new messages, updating statuses). They interact with the database through the ORM layer.
    - UserController, ProfileController: Standard HttpServlet classes handle specific HTTP requests, primarily for user registration and profile image uploads.

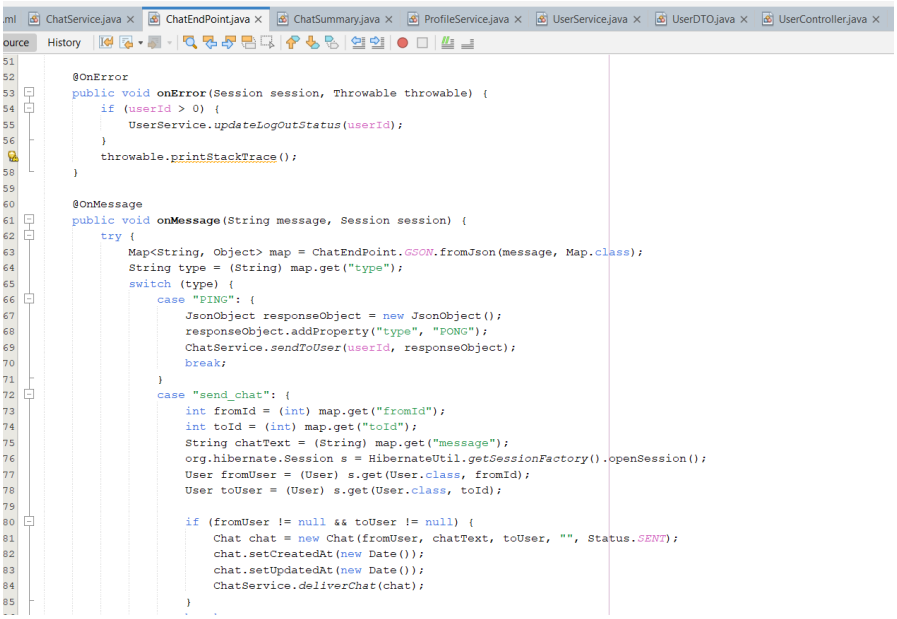


Figure 2:backendcode

3. Communication Layer:
    - Technology: Utilizes the WebSocket protocol for the primary communication channel.
    - Role: Provides a persistent, full-duplex communication link between the mobile app client and the Java backend server. This is crucial for delivering the real-time nature of the chat, allowing instant message delivery, status

updates (online/offline, typing indicators, read receipts), and immediate synchronization of the contact list. Initial authentication or non-real-time actions might use standard HTTP requests (Fetch API), but ongoing chat interaction relies on the persistent WebSocket connection established to the ChatEndPoint.

4. Database Layer:
    - Technology: Uses a MySQL relational database, managed by the Hibernate ORM framework.
    - Role: This layer is responsible for the persistent storage of all application data.
    - Data Stored: Includes user account details (username, password hash, name, contact info, profile image path), contact/friend relationships, and the entire history of chat messages exchanged between users.
    - Interaction: The Hibernate ORM abstracts the raw SQL interactions, mapping Java objects (like User, Chat, FriendList entities) to database tables and vice-versa, simplifying data access and manipulation within the Java backend services (UserService, ChatService).

Features

- User Authentication & Onboarding:
    - User Registration: New users can create an account by providing their first name, last name, username, country code, phone number, and selecting/uploading a profile image.
    - User Login: Registered users can log in using their username and password.
    - Persistent Login Session: User sessions are persisted using AsyncStorage, allowing users to remain logged in after closing and reopening the app.
    - Onboarding Flow: A guided introduction for new users, consisting of three informative screens explaining the app's core concepts.
- Real-Time Communication:
    - Instant Messaging: Exchange text messages instantly with contacts using a persistent WebSocket connection.
    - Message Status Indicators: Visual feedback on sent messages showing their status: SENT, DELIVERED, and READ, including dedicated icons/checkmarks.

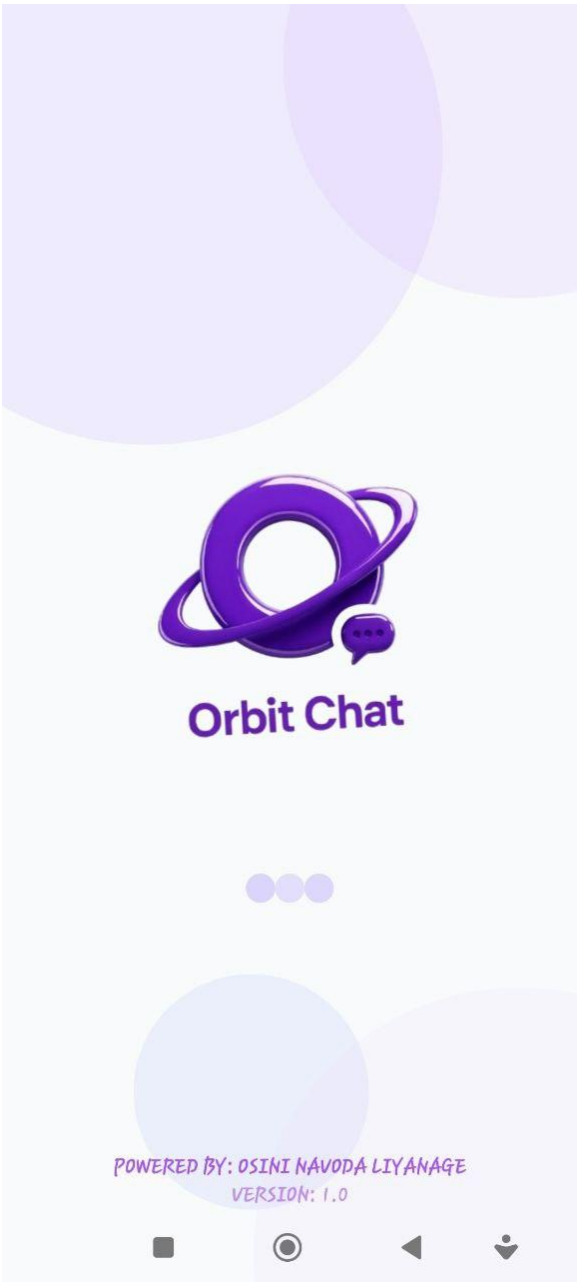
    - Online/Offline Status: See the real-time online/offline status of contacts directly in the chat list and chat window header.
    - Real-Time Chat List Updates: The main chat list automatically refreshes to show the latest conversations, unread message counts, and updated timestamps as new messages arrive or statuses change.
- Contact Management:
    - Add New Contacts: Search for and add new contacts using their registered phone number.
    - Contact List: View a list of all added contacts, including their profile pictures, names, and online status.
- Messaging Interface:
    - Individual Chat Windows: Dedicated screens for private conversations with a specific contact, displaying the message history.
    - Message History: Scroll through the complete chat history with a specific contact within the chat window.
    - Distinct Message Bubbles: Visually differentiated message bubbles for sent messages (purple) and received messages (white/grey).
    - Message Timestamps: Precise timestamps displayed under each message.
    - Unread Message Counters: Numerical badges indicating the number of unread messages per conversation in the main chat list.
- User Profile & Settings:
    - View Profile: Users can view their own profile information (name, phone number, profile image) on a dedicated profile screen.
    - Edit Profile Picture: Ability to select a new profile image from the device's gallery.
    - Theme Customization: Users can choose their preferred app theme: Light, Dark, or System default, with full support throughout the application.
    - Logout Functionality: Securely log out of the current session, clearing user data and returning to the login screen.
- User Experience Enhancements:
    - Intuitive UI/UX Design: A clean, modern interface featuring a distinctive cosmic purple color scheme and gradient headers.
    - Responsive Design: Adaptable layouts ensuring usability across various screen sizes and orientations.

- Smooth Animations: Subtle animations for screen transitions, element appearances, and interactive elements (e.g., buttons, search bars) enhancing the overall feel.
- User Feedback Alerts: Informative alerts and notifications for actions like successful login/logout, message sending, errors, or validation warnings.
- Cross-Platform Compatibility: Built with React Native, ensuring consistent functionality and appearance on both Android and iOS devices.

User Interface (UI) Design

1. SplashScreen.tsx

- Purpose: The initial loading screen that appears when the app starts.
- Functionality: Displays the Orbit Chat logo with animated orbs and loading dots. It serves as a brand introduction and gives the app time to initialize (e.g., check for saved user session).
- Design: Features a dark background with floating purple orbs and the app logo centered. A progress indicator (dots) shows the app is loading. Use animateing parts.
- Navigation: Automatically navigates to the Onboarding or Home screen after a set time.



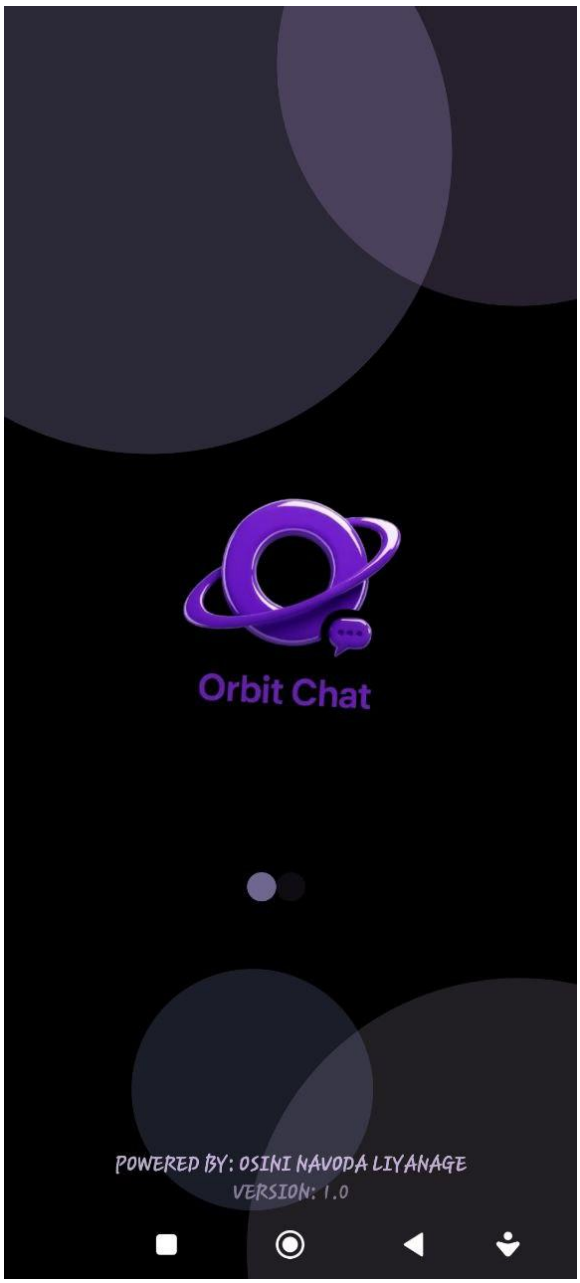


Figure 3:Splashscreen2

Figure 4:SplashScreen1

2. OnboardingScreen1.tsx**,** OnboardingScreen2.tsx**,** OnboardingScreen3.tsx

- Purpose: Guide new users through the app's key features before they start using it.
- Functionality: Each screen presents a different aspect of the app (e.g., "Simple & Easy to Use," "Discover New Friends," "Secure & Private"). Users swipe or tap "Next" to proceed.
- Design: Clean, modern design with illustrations and concise text. Uses a dot indicator to show progress through the onboarding flow.
- Navigation: Navigates sequentially from 1 to 3, then to the Sign Up or Login screen.

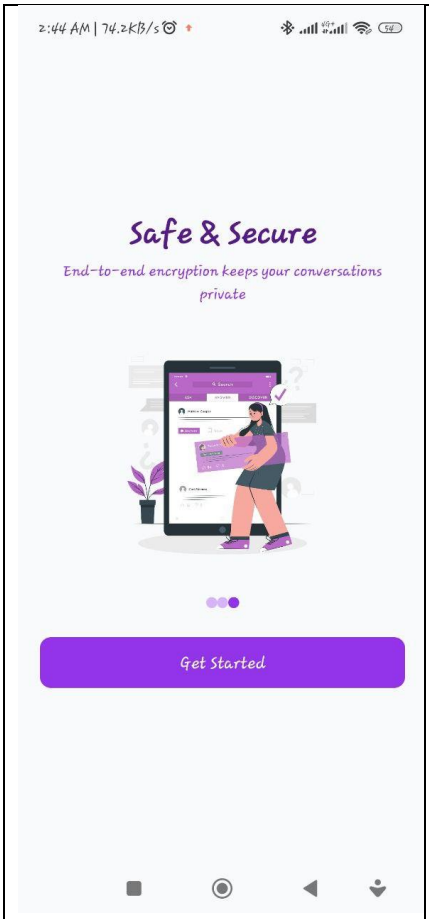


Figure 5:OnbordingScreen1

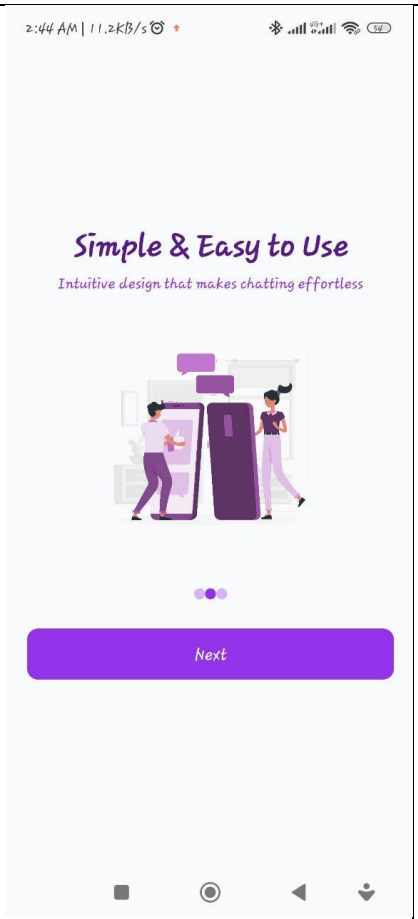


Figure 6:onbordingscreen2

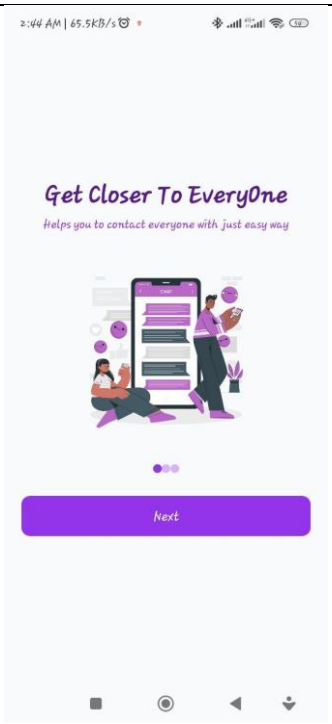


Figure 7:onbordingscreen3

3. SignUpScreen.tsx

- Purpose: Allows new users to create an account.
- Functionality: Collects user information like First Name, Last Name, Username, Country Code, and Phone Number. Optionally allows profile picture upload.
- Design: Form-based layout with input fields and a prominent "Continue" button. Uses a purple gradient theme.
- Navigation: After successful registration, navigates to contact screen , then AvatarScreen to set up the profile.

Figure 8:signUp screen1

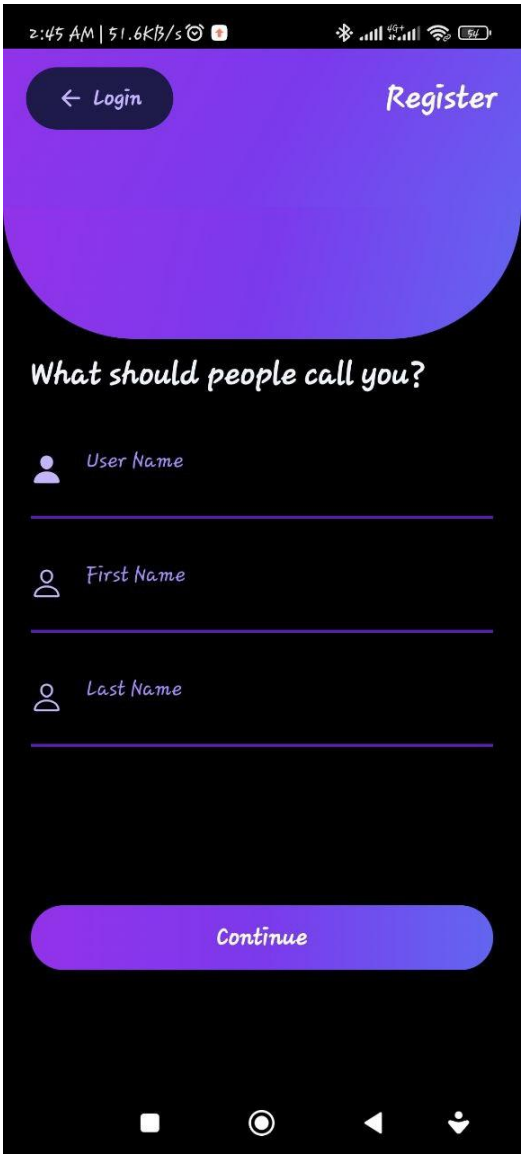
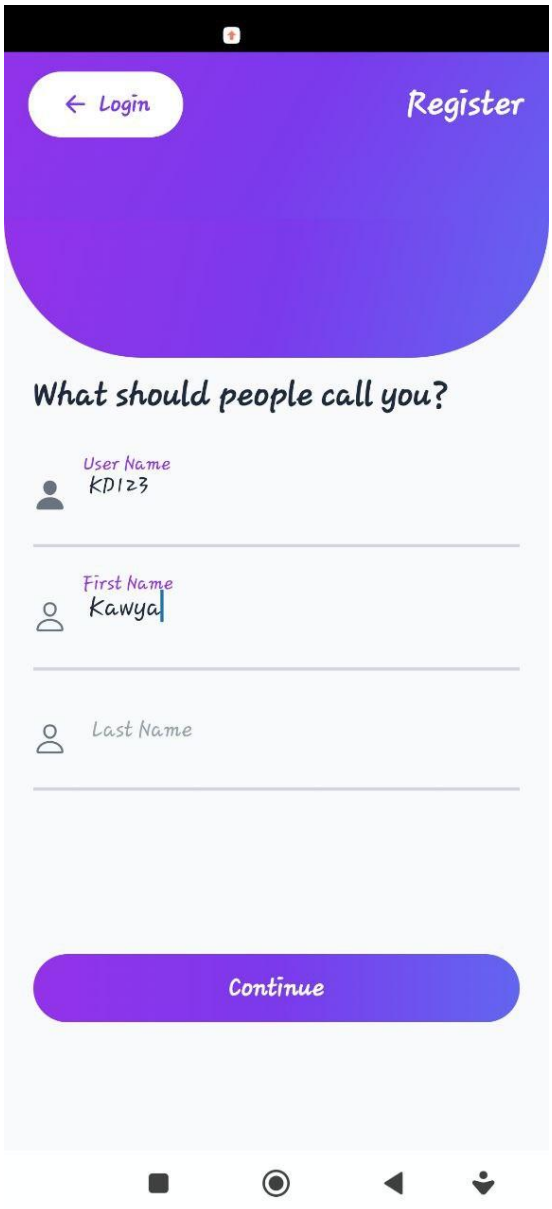


Figure 9:signupscreen2



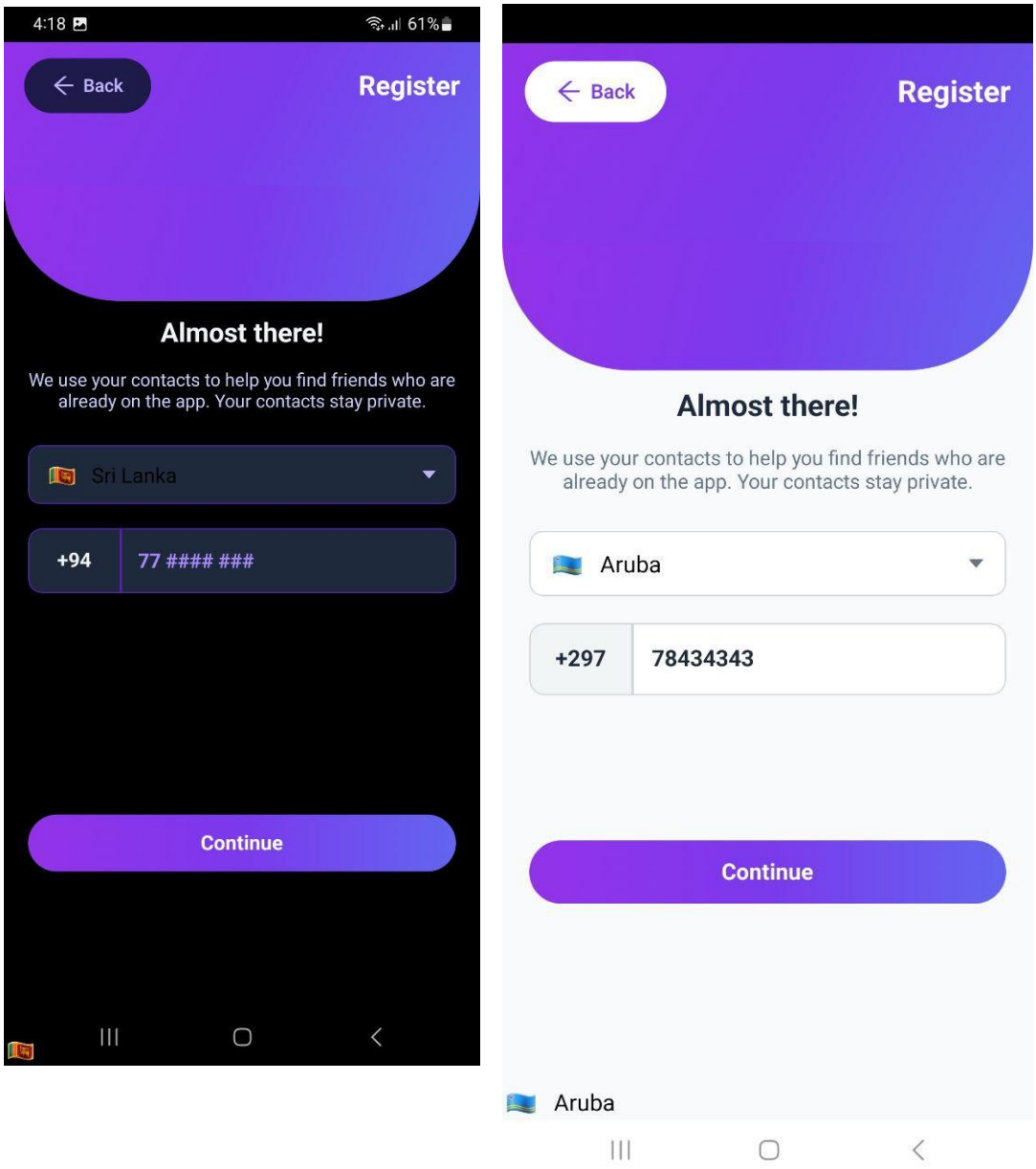


Figure 10:contactscreen2

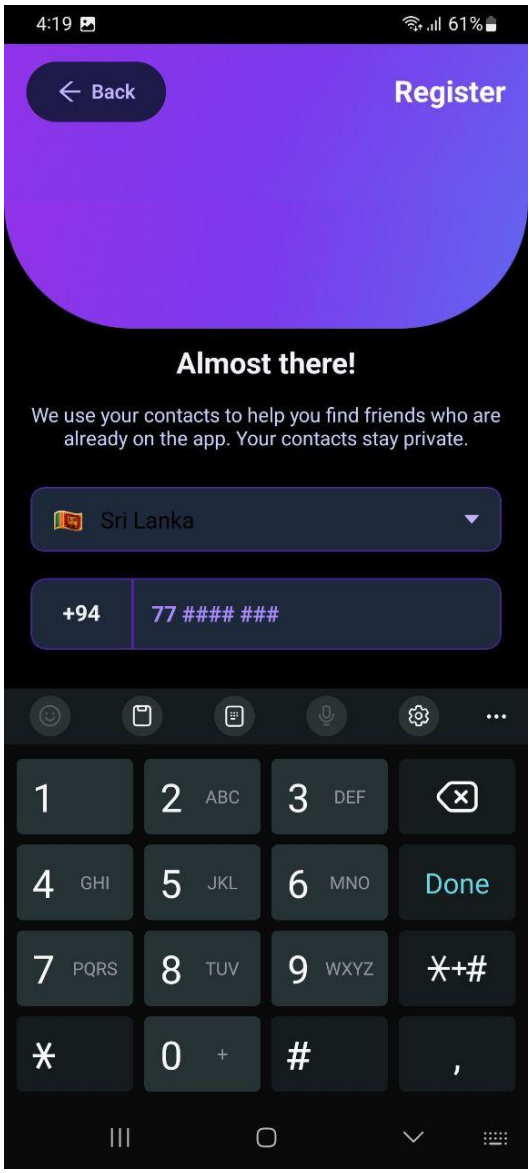


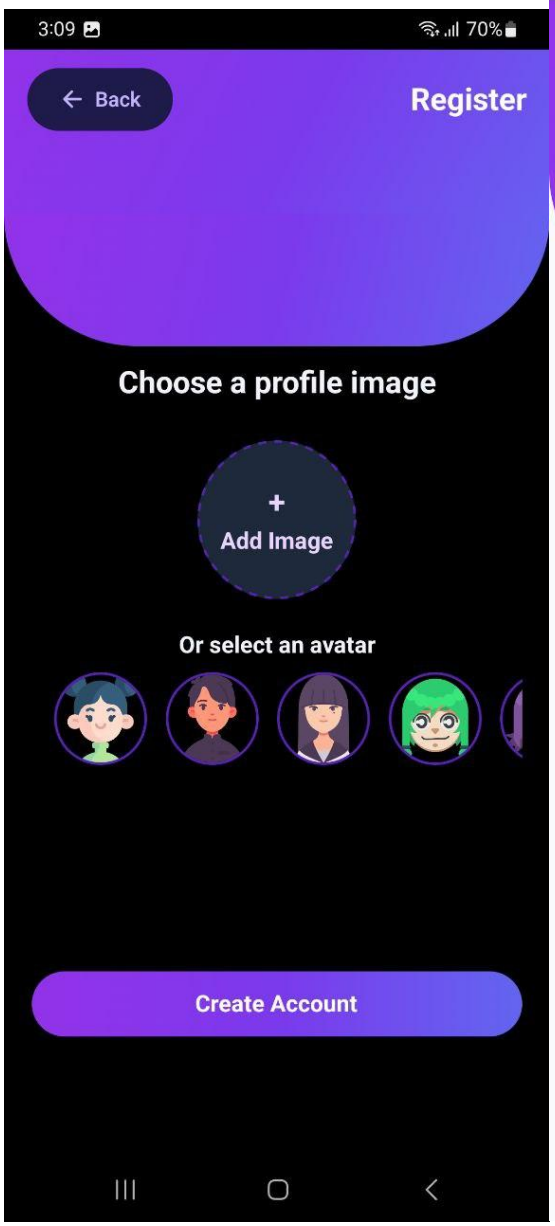
Figure 11:contactscreen3

Figure 12:contactscrren1

4. AvatarScreen.tsx

- Purpose: Allows users to select or upload a profile picture after signing up.

- Functionality: Presents a grid of default avatars or an option to pick an image from the device gallery. Saves the selected image to the user's profile.
- Design: Centered profile image placeholder with options below. Clean and simple interface.
- Navigation: After selecting an avatar, navigates to HomeScreen.



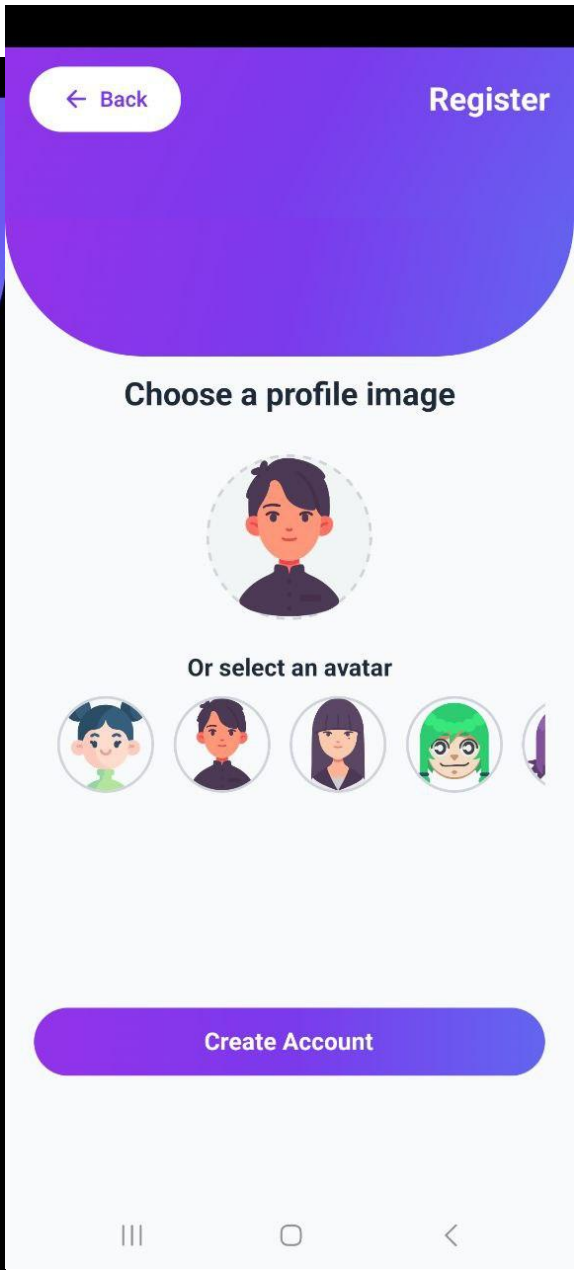


Figure 13:avatarscrenn2

Figure 14:avatar screen1

5. SignInScreen.tsx

- Purpose: Allows existing users to log into their account.
- Functionality: Requires the user to enter their username and password. Validates credentials against the backend.
- Design: Simple form with username/password fields and a "Login" button. Includes a link to "Sign Up" for new users.
- Navigation: On successful login, navigates to HomeScreen. If login fails, displays an error message.

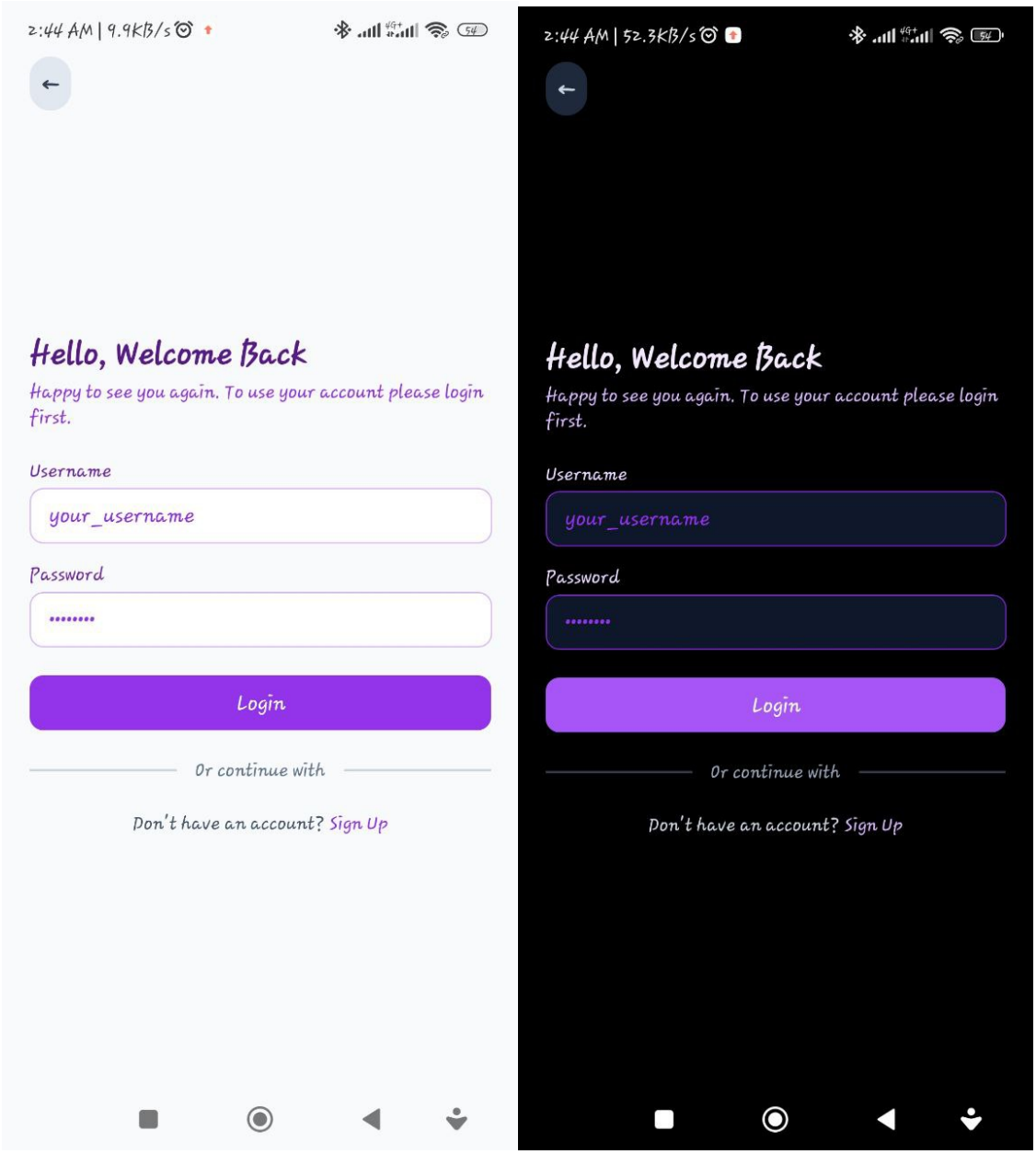


Figure 15:signinscreen2

Figure 16:signinscreen

6. HomeScreen.tsx - ChatsScreen.tsx**)**

- Purpose: The main hub of the app, displaying a list of recent conversations.
- Functionality: Shows a list of contacts with their latest message, timestamp, and online status. Includes a search bar for finding specific chats. Tapping a contact opens the individual chat window.
- Design: Features a purple header with the app name and search icon. Messages are displayed in a list with avatars and message previews.
- Navigation: Tapping a contact navigates to SingleChatScreen. Tapping the bottom tab bar icons navigates to ContactScreen or ProfileScreen.

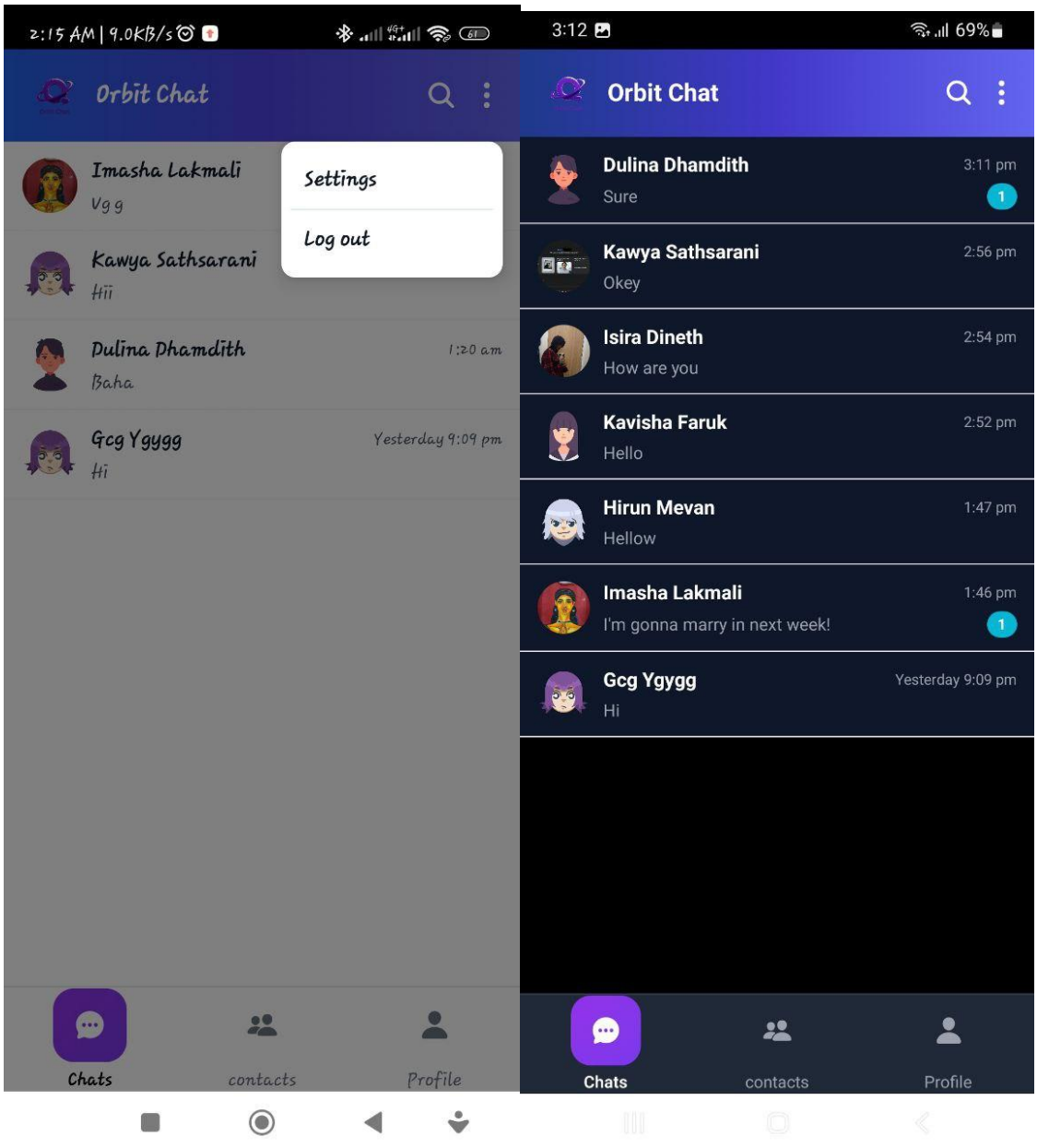
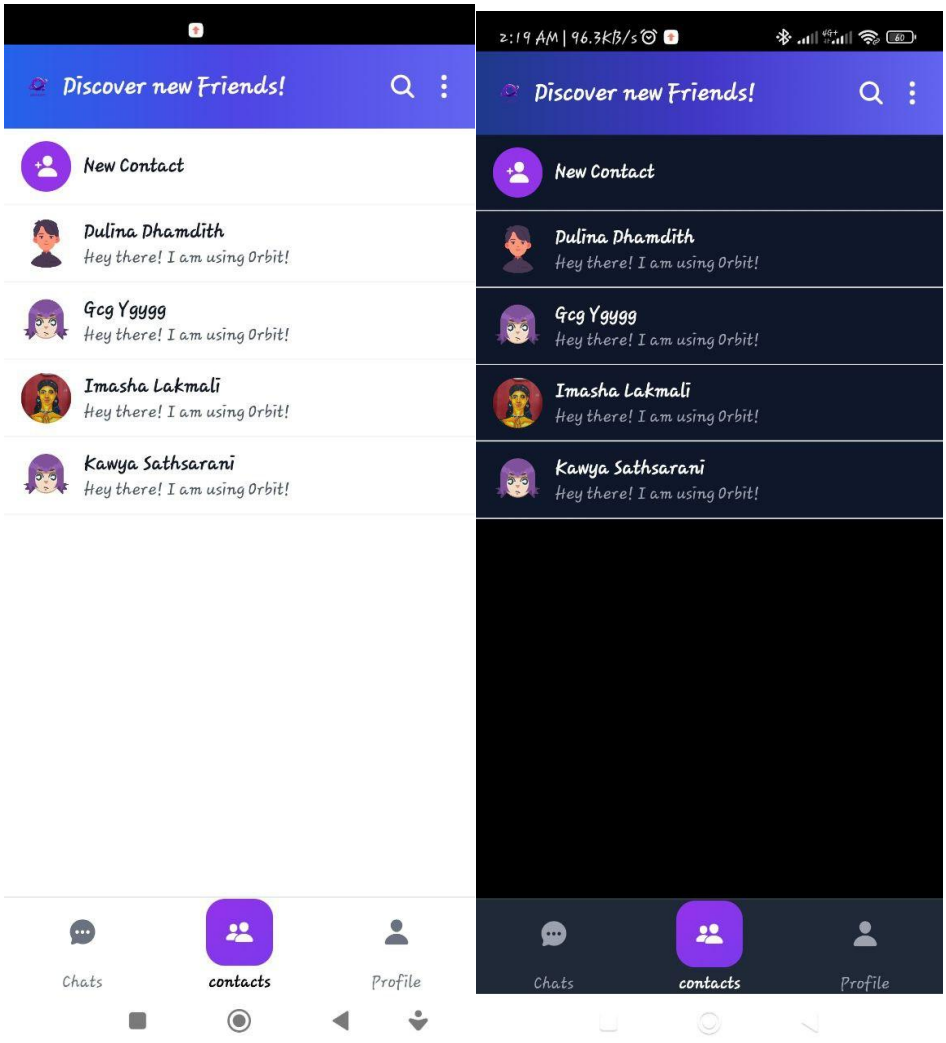


Figure 17:chatscrren

Figure 18:chatappmodel



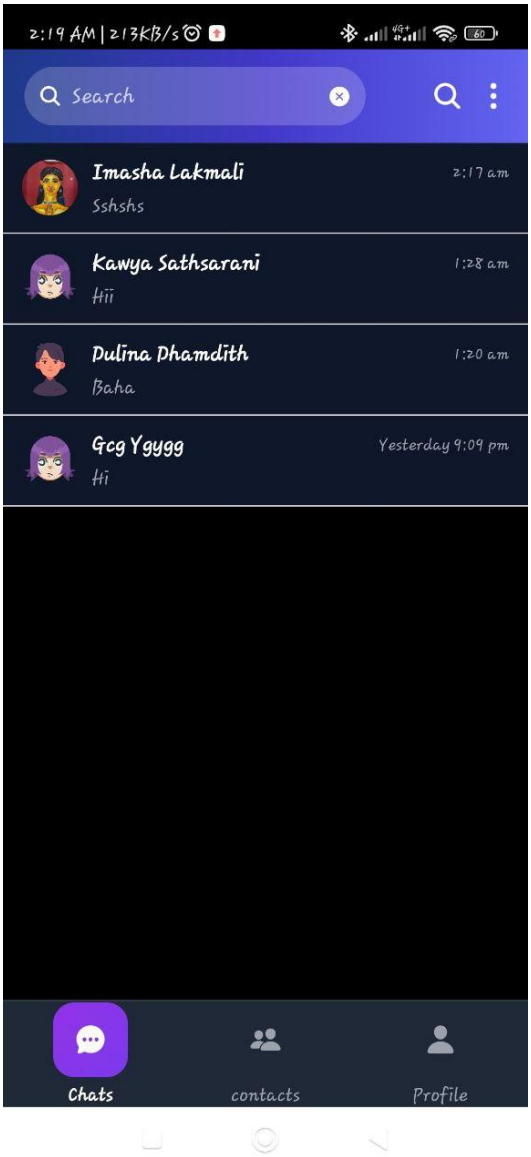


Figure 19:home contact screen2

Figure 20:homecontactscreen1

8. SingleChatScreen.tsx

- Purpose: The individual chat window for private messaging with a specific contact.
- Functionality: Displays the full conversation history with a specific contact. Allows users to send new messages. Shows message delivery/read status.
- Design: Header with contact name and online status. Message bubbles for sent (purple) and received (white) messages. Input field at the bottom for typing new messages.
- Navigation: Tapping back returns to HomeScreen.

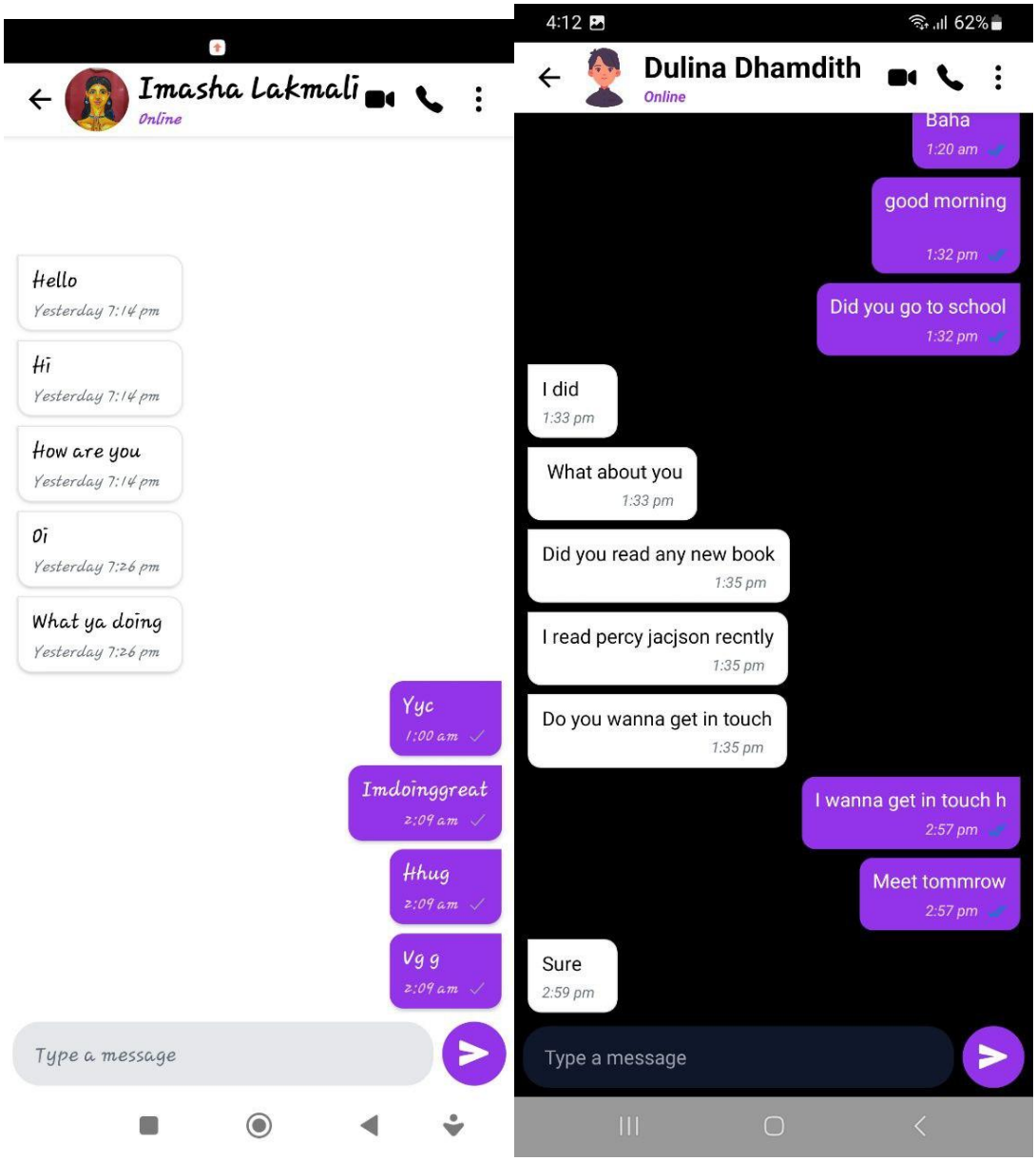


Figure 21:singlechatscreen1

9. ProfileScreen.tsx

- Purpose: Displays the user's own profile information.
- Functionality: Shows the user's name, phone number, and profile image. May include options to edit the profile or logout.
- Design: Centered profile image with user details below. Clean, card-based layout.
- Navigation: Tapping "Edit Profile" might navigate to AvatarScreen or a dedicated edit screen. Tapping "Logout" logs the user out.

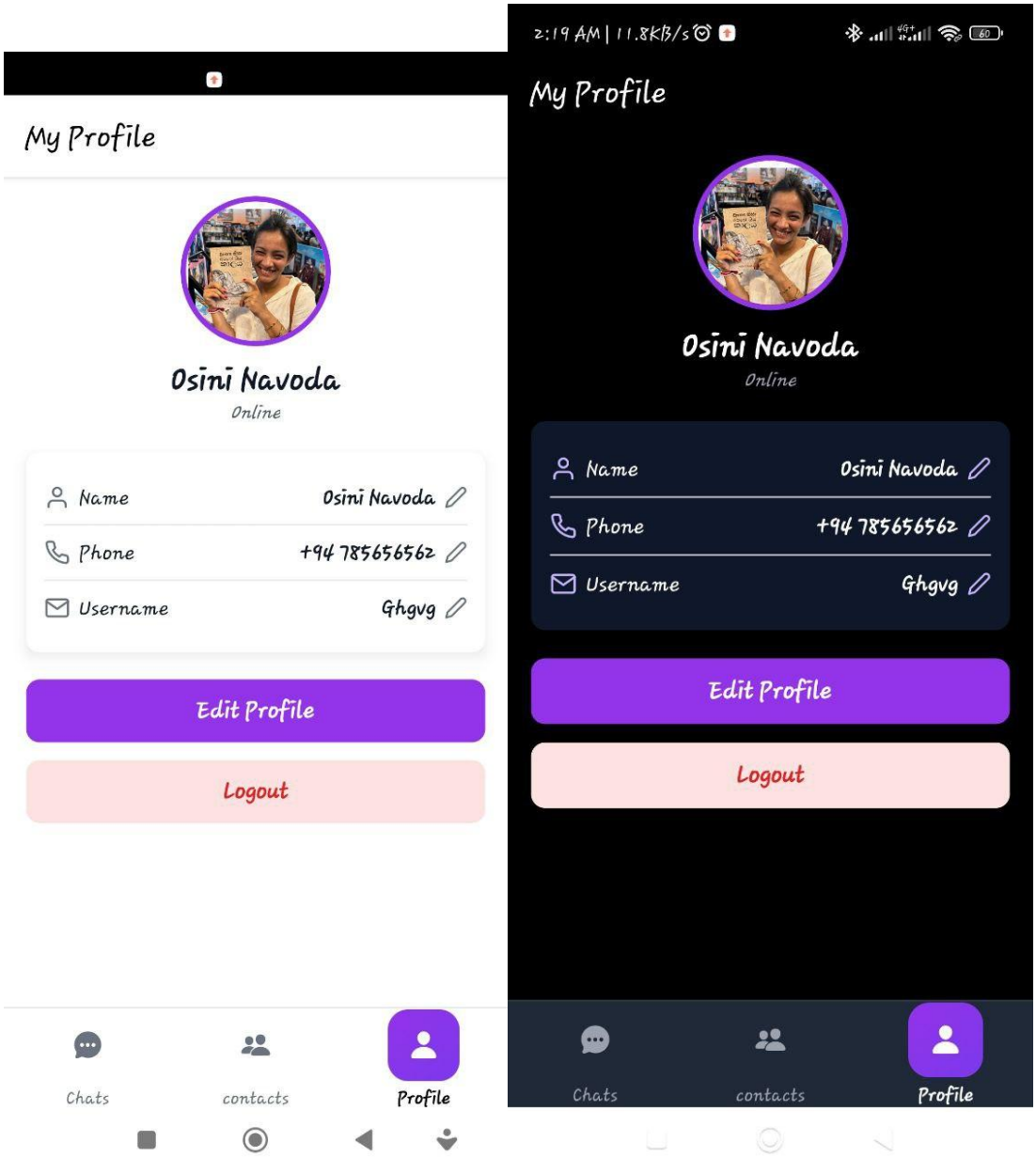


Figure 22:profilescreen2

Figure 23:profilesceen1

10. SettingScreen.tsx

- Purpose: Allows users to customize app settings.
- Functionality: Includes options for changing the app theme (Light/Dark/System), managing notifications, viewing privacy policy, and logging out.
- Design: Organized into sections with clear headings and toggle buttons or links.
- Navigation: Tapping "Logout" logs the user out. Tapping "Privacy Policy" navigates to PrivacyScreen.

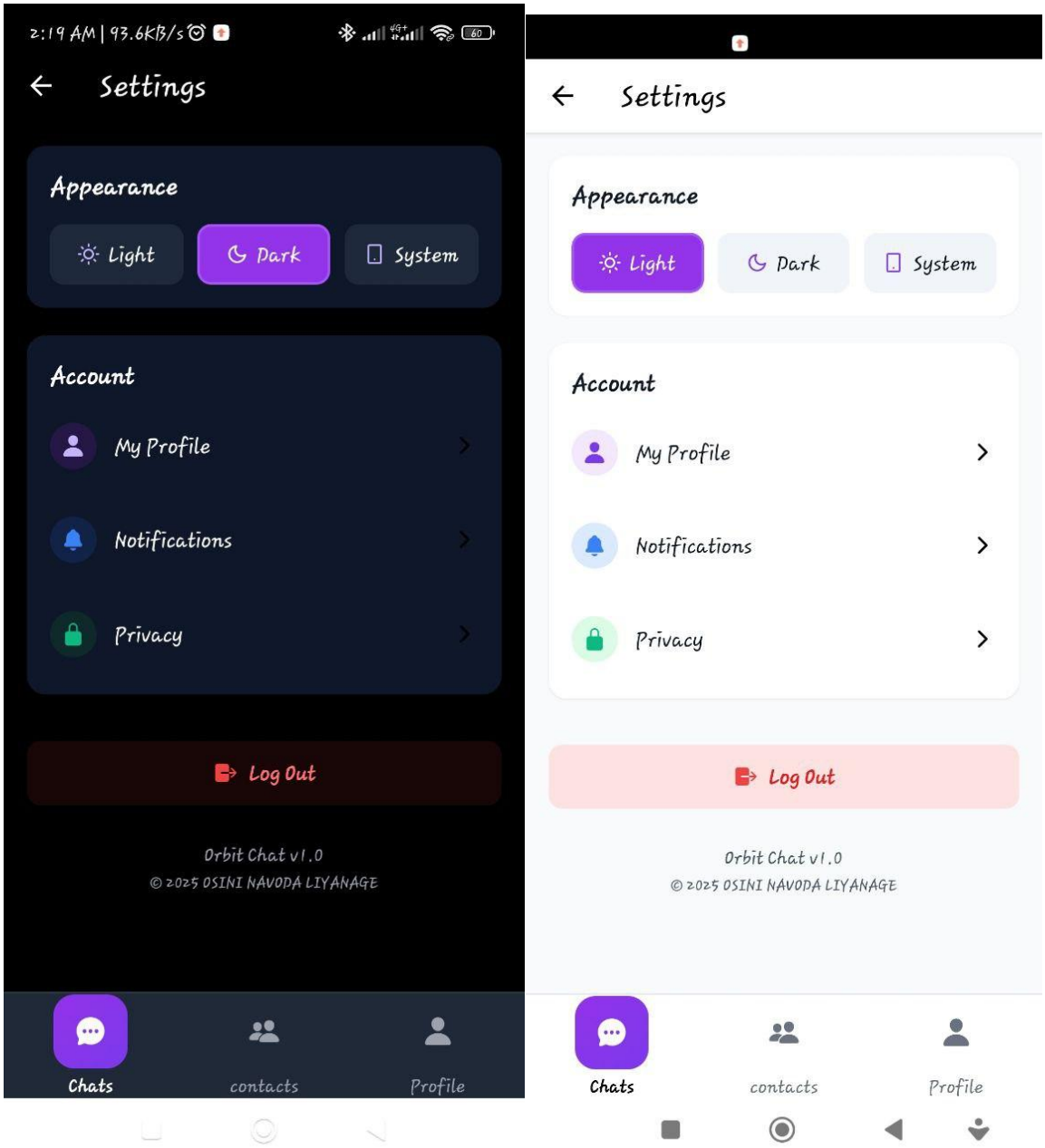


Figure 24:setting screen 2

Figure 25:setting sceen1

11. PrivacyScreen.tsx

- Purpose: Displays the app's privacy policy.
- Functionality: Presents a detailed document outlining what data is collected, how it's used, and user rights under GDPR.
- Design: Text-heavy screen with clear headings and bullet points. Uses a dark theme for readability.
- Navigation: Tapping "Back to Settings" returns to SettingScreen.

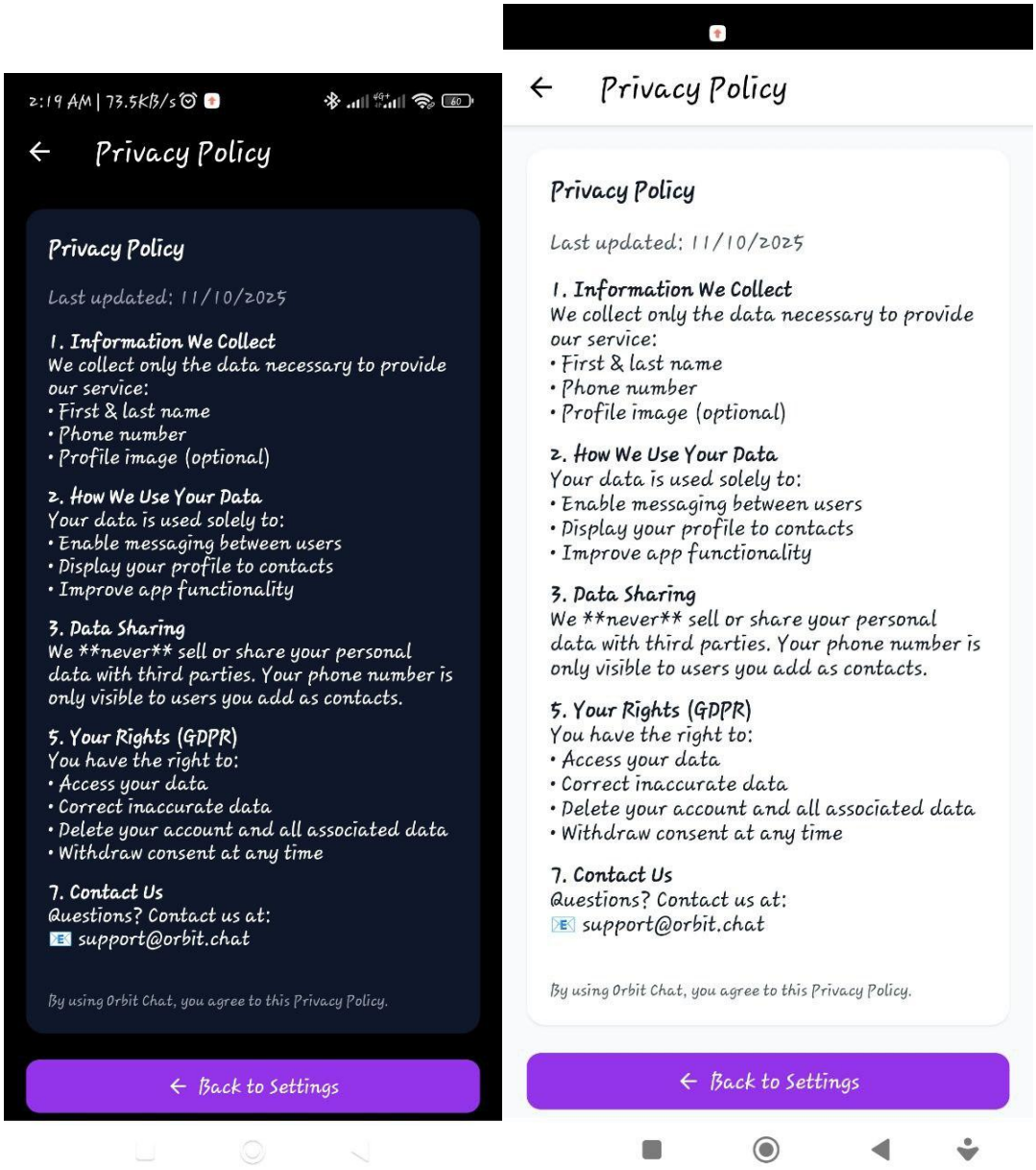


Figure 26:privacyscreen2

Figure 27:privacy screen1

12. SignOut.tsx **(likely part of** SettingScreen**)**

- Purpose: Handles the user logout process.
- Functionality: Clears the user's session data (e.g., userId from AsyncStorage) and navigates them back to the login screen.
- Design: Often implemented as a modal or a button within SettingScreen rather than a separate screen.

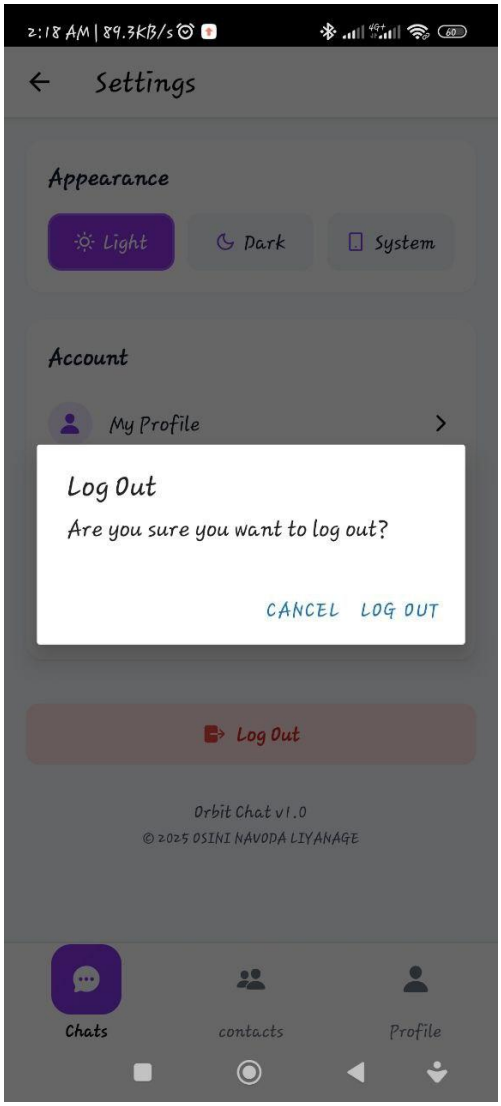
- 

Figure 28:signout

13. Home ContactScreen.

- Purpose: Allows users to add new contacts to their list.
- Functionality: Provides a form to enter a contact's First Name, Last Name, Country, and Phone Number. Searches for the contact in the system and adds them if found.
- Design: Form with input fields and a "Save Contact" button. Uses the same purple theme as other screens.
- Navigation: After saving, returns to the previous screen (often HomeScreen).

Figure 29:chatscreenwith searchbaropen

Figure 30:new contactscrren2

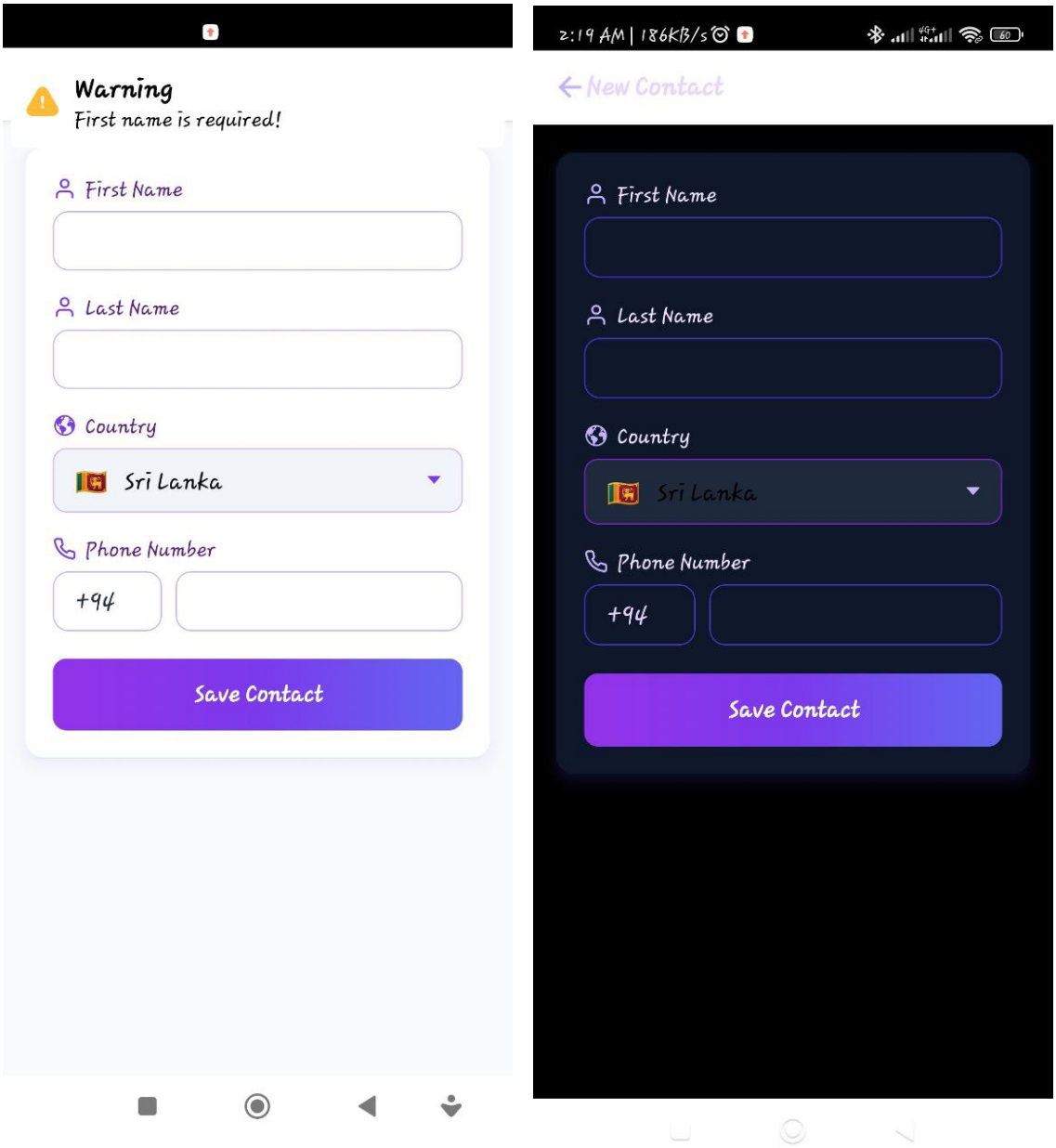


Figure 31:newcontactscreen1

14.notification screen

Ths one only have design for now. I hope to add backend in the future enhacment.

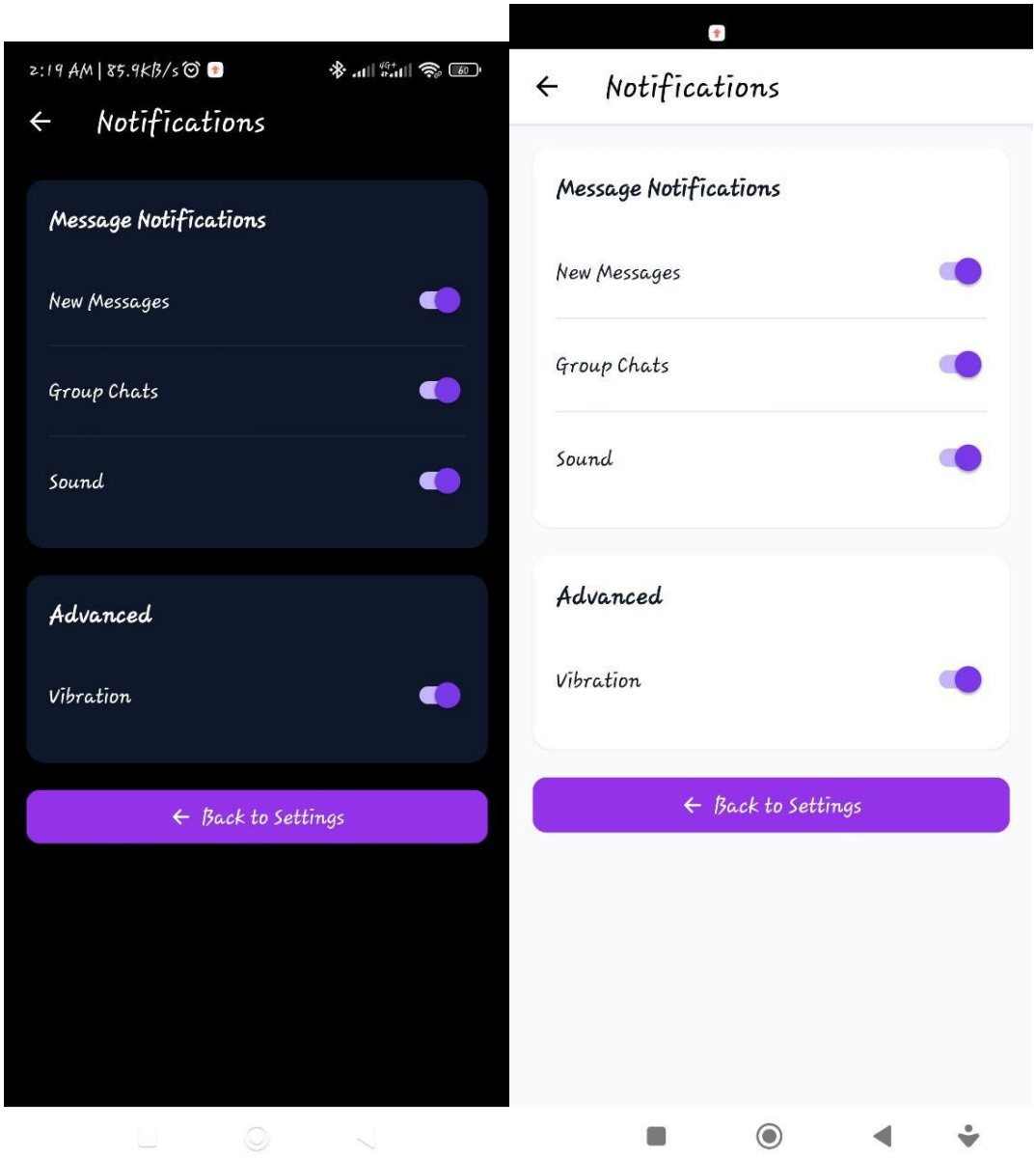


Figure 32:notificationscreen2

Figure 33:notification screen1

Diagrams

This section contains visual diagrams for better understanding:

- ER Diagram (Entity-Relationship):

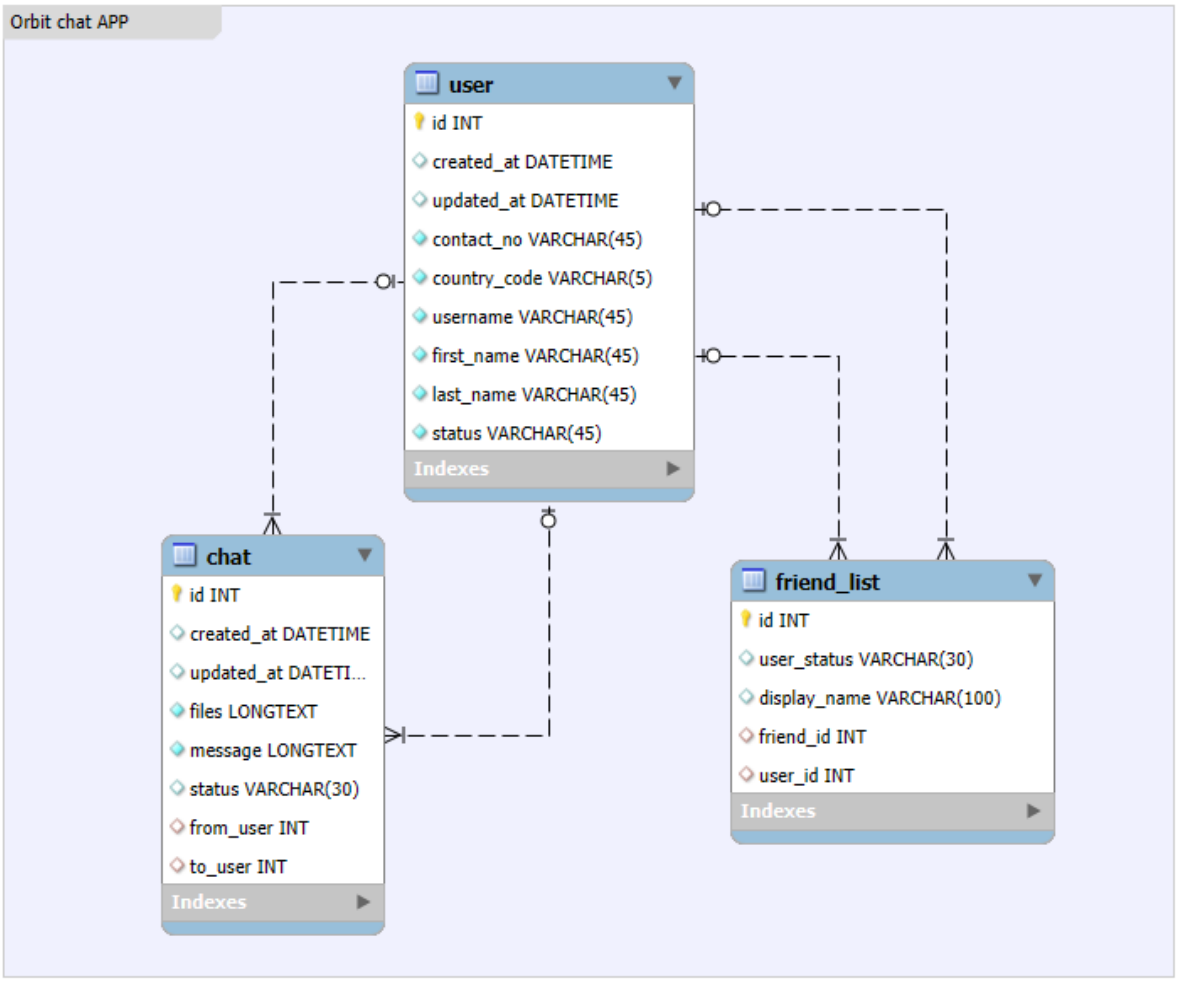


Figure 34:ER DIGRAME

- Use Case Diagram:

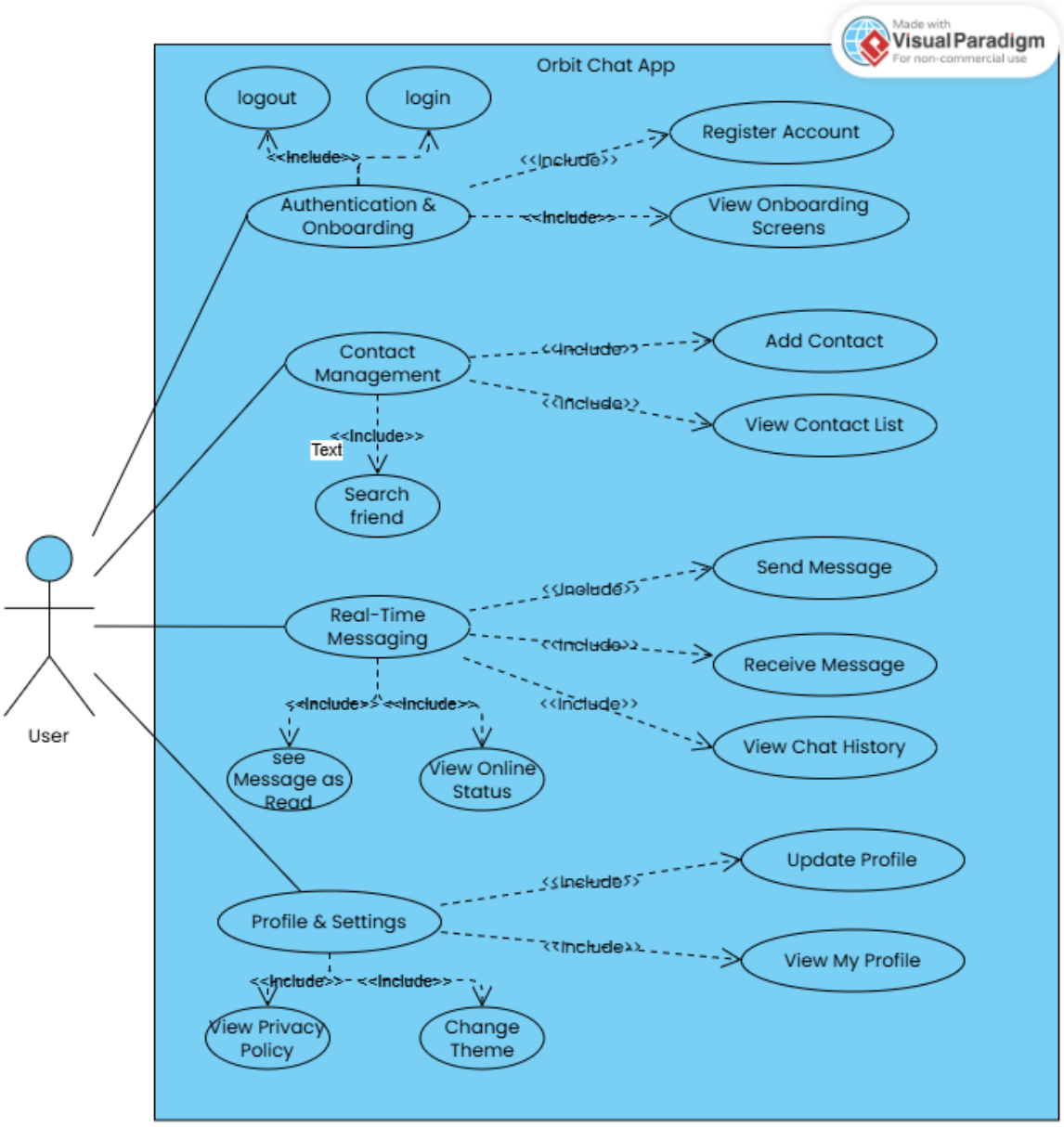


Figure 35:USECASE DIGRAM

- Class Diagram:

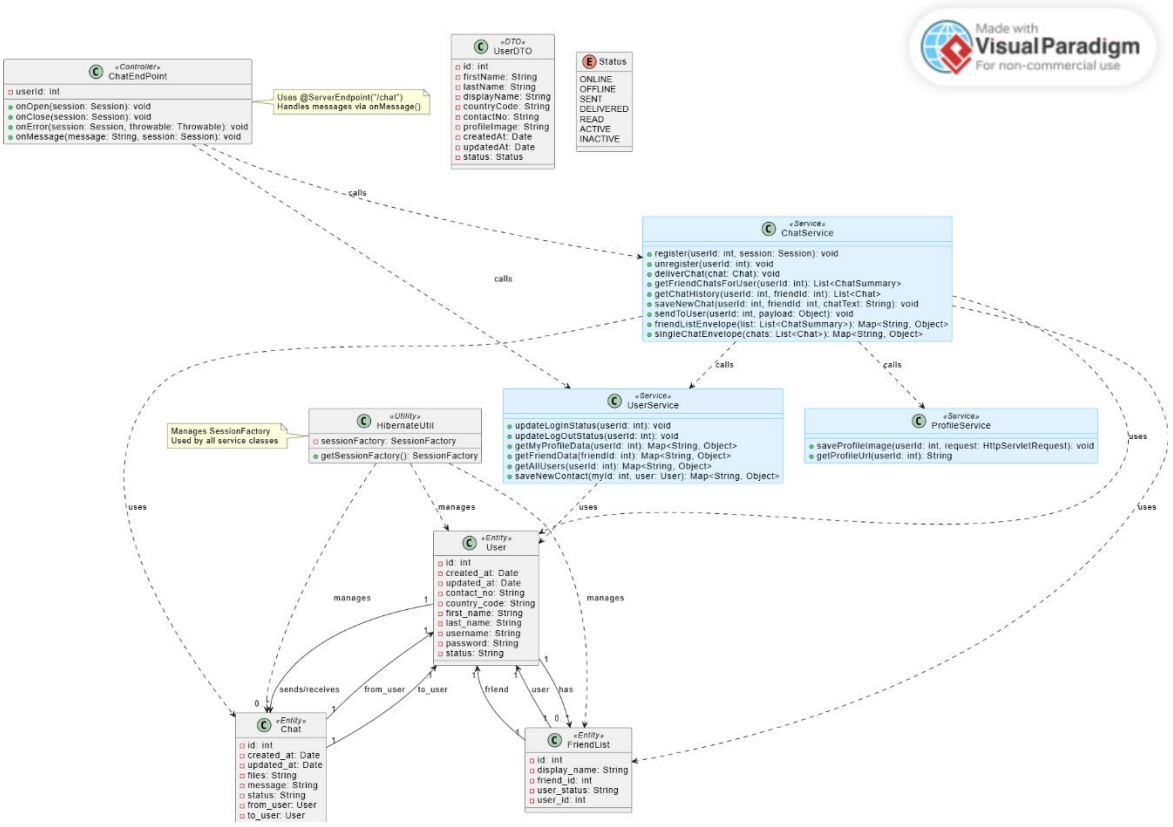


Figure 36:CLASS DIGRAM

Logo

Figure 37:ORBIT-LOGO

Backend Implementation

The Orbit Chat backend is built using Java EE (Servlet API) with Hibernate ORM for database interactions and WebSocket for real-time communication. The system handles user authentication, chat messaging, and contact management through a combination of RESTful endpoints and persistent WebSocket connections.

1. User Authentication & Registration

- Signup (UserController):
    - A POST request to /UserController receives user data: userName, firstName, lastName, countryCode, contactNo, and profileImage.
    - It validates that all fields are present.
    - Checks if a user with the same countryCode and contactNo already exists.
    - If not, it creates a new User object, sets the password to "password" (for now), saves it to the database, and saves the profile image to the server's profile-images folder.
    - Returns a JSON response indicating success or failure.
- Login (Not fully implemented in provided code, but implied):
    - The ChatEndPoint expects a userId parameter in the WebSocket connection URL (?userId=...).
    - This suggests a separate login endpoint (e.g., LoginController) would validate credentials and return the userId to the client, which then uses it to establish the WebSocket connection.
    - Upon successful login, the onOpen method in ChatEndPoint registers the user’s session and updates their status to ONLINE.

2. Database Management with Hibernate

- ORM Mapping: Hibernate maps Java objects (User, Chat, FriendList) to relational database tables.
- Entities: Key entities include:
    - User: Stores user information (id, name, contact, status, etc.). Extends BaseEntity for automatic createdAt and updatedAt timestamps.
    - Chat: Represents a single message with from and to user references, message text, and status (SENT, DELIVERED, READ).

    - FriendList: Manages the relationship between users (who is friends with whom) and their display names.
    - Status: An enum for user and message statuses.
- CRUD Operations: Services like UserService and ChatService use Hibernate's Session and Criteria to perform database operations (create, read, update, delete).

3. Real-Time Communication via WebSocket

- Endpoint: The @ServerEndpoint(value = "/chat") annotation defines the WebSocket endpoint.
- Connection Handling:
    - onOpen: Called when a client connects. Extracts userId from the query string, registers the session, and updates the user's status to ONLINE.
    - onClose: Called when a client disconnects. Unregisters the session and updates the user's status to OFFLINE.
    - onError: Logs errors and ensures the user's status is set to OFFLINE on error.
- Message Handling (onMessage):
    - Parses incoming JSON messages.
    - Routes messages based on the type field:
        - send_chat / send_message: Saves a new message to the database and delivers it to the recipient(s) via ChatService.deliverChat() or ChatService.saveNewChat(). Updates the status of messages as they are delivered/read.
        - get_chat_list: Sends the list of recent conversations to the user.
        - get_single_chat: Fetches the full history of a specific conversation and sends it to the user.
        - get_friend_data: Sends data about a specific friend.
        - get_all_users: Sends a list of all users (for adding contacts).
        - save_new_contact: Adds a new contact to the user's friend list.
        - set_user_profile: Sends the current user's profile data.
        - PING: Responds with a PONG to keep the connection alive.

4. REST APIs for Non-Real-Time Data

- UserController: Handles user registration via HTTP POST.
- ProfileController: (Implied by uploadProfileImage function) Handles uploading and saving profile images.

- LoginController: (Implied) Would handle user login via HTTP POST and return the userId for WebSocket connection.

5. Profile Image Handling

- Upload: Profile images are uploaded via multipart/form-data to the UserController or ProfileController.
- Storage: Images are saved to the server's profile-images folder, organized by user ID.
- Access: Profile images are served via a direct URL (e.g., https://056a5d00dac4.ngrok-free.app/OrbitBackend/profile-images/{userId}/profile1.png).

Frontend Implementation

The Orbit Chat frontend is built using React Native (Expo), providing a native-like experience on both Android and iOS. The UI is designed with a modern, clean aesthetic using NativeWind (Tailwind CSS) for consistent styling. State management and real-time communication are handled through custom hooks and WebSocket integration.

1. React Native Components & UI

- Core Components: The app uses standard React Native components (View, Text, Image, TextInput, TouchableOpacity, FlatList, Modal) to build the UI.
- Screen Layouts:
    - Splash/Onboarding: Full-screen layouts with animated logos, orbs, and progress indicators.
    - Login/Signup: Form-based screens with input fields, floating labels, and gradient buttons.
    - Home Screen: A list of conversations with avatars, names, message previews, and timestamps.
    - Single Chat Screen: A dedicated chat window with a header showing contact info, a scrollable message area with distinct bubbles for sent/received messages, and an input bar at the bottom.
    - Profile/Settings: Card-based layouts for displaying user info and settings options.
- Chat Bubbles: Messages are displayed in rounded rectangular bubbles. Sent messages are styled with a purple background (bg-purple-600), while received messages use a white/grey background (bg-white or bg-slate-900 in dark mode).
- Navigation: Uses @react-navigation/native with a NativeStack for screen transitions and a bottom tab bar for switching between main sections (Chats, Contacts, Profile).

2. State Management

- Context API: Global state like authentication status (userId) is managed using AuthContext and UserRegistrationProvider.

React Hooks Used

1. useState

- What it does: Declares a state variable and a function to update it.
- Used for: Managing local component state (e.g., input, searchVisible, image, messages).
- Where used: In every screen (HomeScreen.tsx, SingleChatScreen.tsx, SplashScreen.tsx, AvatarScreen.tsx, etc.).

2. useEffect

- What it does: Runs side effects after render (e.g., data fetching, subscriptions, manual DOM manipulations).
- Used for: Setting up animations, connecting to WebSocket, loading data, cleaning up resources.
- Where used: In SplashScreen.tsx (animations), HomeScreen.tsx (search animation), SingleChatScreen.tsx (WebSocket setup).

3. useLayoutEffect

- What it does: Similar to useEffect, but fires synchronously after all DOM mutations. Use it to read layout from the DOM and synchronously re-render.
- Used for: Updating navigation options (like header title) that depend on layout (e.g., when a component mounts or updates).
- Where used: In HomeScreen.tsx, SingleChatScreen.tsx, ProfileScreen.tsx (to set dynamic header titles).

4. useRef

- What it does: Creates a mutable ref object whose .current property is initialized to the passed argument (initialValue). The returned object will persist for the full lifetime of the component.
- Used for: Holding mutable values that don't trigger re-renders (e.g., Animated.Value instances for animations).
- Where used: In HomeScreen.tsx, SignInScreen.tsx, SplashScreen.tsx (for Animated.Value).

5. useMemo

- What it does: Memoizes a value. Recalculates the value only if its dependencies change.
- Used for: Optimizing performance by avoiding expensive calculations on every render.
- Where used: In AuthProvider.tsx (to memoize the value object for AuthContext).

Custom Hooks: Custom hooks handle specific state:

    - useTheme(): Manages light/dark mode state and provides theme colors.
    - useChatList(), useSingleChat(), useUserProfile(): Fetch and manage chat and user data from the backend via WebSocket.
    - useSendChat(), useSendNewContact(): Handle sending new messages or contacts.
- Local State: Component-specific state (e.g., search text, form inputs, modal visibility) is managed with useState.

3. WebSocket Integration

- Connection: A WebSocketProvider establishes a persistent connection to the Java backend's /chat endpoint, passing the userId as a query parameter.
- Real-Time Updates:
    - Message Delivery: When a message is sent, it’s dispatched via the WebSocket. The backend delivers it to the recipient, who receives it in real-time.
    - Status Updates: The backend updates the status of messages (SENT → DELIVERED → READ) and sends these updates back to the client via the WebSocket.
    - Friend List Sync: When a new contact is added or a message is sent, the backend pushes an update to the client to refresh the friend list.
- Hooks: Custom hooks like useChatList and useSingleChat listen for incoming WebSocket messages and update the UI accordingly.

4. Animations & Alerts

- Animations: Uses react-native-reanimated for smooth, performant animations:
    - Entrance Animations: Logo scaling and fading in on splash/login screens.
    - Search Bar: Expands and contracts when toggled.
    - Loading Dots: Pulsing animation on the splash screen.

    - Button Presses: Subtle scale-down effect on press.
    - Orbs: Gentle pulsing in the background.
- Alerts: Uses react-native-alert-notification to display:
    - Validation errors (e.g., "Username is required").
    - Success/error messages (e.g., "Account created", "Login failed").
    - Confirmation dialogs (e.g., "Log out?").

Database Design

The Orbit Chat application uses a relational database (MySQL) to store user, chat, and contact data. The schema is designed with three core tables: user, chat, and friend_list, which are interconnected to support the app's functionality.

1. user **Table**

This table stores information about each registered user.

- Primary Key: id (INT)
- Columns:
    - created_at: DATETIME (Timestamp of account creation)
    - updated_at: DATETIME (Timestamp of last update)
    - contact_no: VARCHAR(45) (User's phone number)
    - country_code: VARCHAR(5) (User's country code, e.g., "+94")
    - username: VARCHAR(45) (Unique username for login)
    - password VARCHAR(45)
    - first_name: VARCHAR(45)
    - last_name: VARCHAR(45)
    - status: VARCHAR(45) (User's online status: "ONLINE", "OFFLINE", "BUSY")

2. chat **Table**

This table stores all messages exchanged between users.

- Primary Key: id (INT)
- Columns:
    - created_at: DATETIME (Message send time)

- updated_at: DATETIME (Last message update time)
- files: LONGTEXT (For storing file paths or metadata; currently unused in your code)
- message: LONGTEXT (The actual text content of the message)
- status: VARCHAR(30) (Message delivery status: "SENT", "DELIVERED", "READ")
- from_user: INT (Foreign key referencing user.id - the sender)
- to_user: INT (Foreign key referencing user.id - the recipient)

3. friend_list **Table**

This table manages the relationships between users (who is friends with whom).

- Primary Key: id (INT)
- Columns:
  - user_status: VARCHAR(30) (Status of the friendship from the perspective of the user_id)
  - display_name: VARCHAR(100) (Custom name for the friend, if set by the user)
  - friend_id: INT (Foreign key referencing user.id - the ID of the friend)
  - user_id: INT (Foreign key referencing user.id - the ID of the user who added the friend)

Relationships

- User to Chat (One-to-Many): A user can send many messages (from_user and to_user foreign keys in chat table).
- User to Friend List (One-to-Many): A user can have many friends (multiple rows in friend_list with the same user_id).

Security Measures

1. Authentication & Session Management

- User Authentication: The app uses a username/password system for login. Upon successful authentication, the backend generates a userId which is used to establish the WebSocket connection.
- Session Persistence: The userId is stored in AsyncStorage using AuthContext, allowing users to remain logged in across app sessions.
- Logout Functionality: When a user logs out, the userId is removed from AsyncStorage, effectively ending the session and requiring re-authentication.

2. Securing WebSocket Connections

- User-Specific Connections: Each WebSocket connection is tied to a specific userId via the query parameter (?userId=...). This ensures that messages are only delivered to the intended recipient's session.
- Connection Validation: The ChatEndPoint validates the userId on connection (onOpen) and updates the user's status accordingly. If userId is invalid or missing, the connection is not registered properly.
- Status Updates: When a user disconnects (or encounters an error), their status is updated to OFFLINE via updateLogOutStatus(userId). This prevents stale connections from being used.

3. Preventing Unauthorized Access to Data

- Server-Side Authorization: All chat-related requests (e.g., get_single_chat, send_message, get_friend_data) are processed by the server, which verifies the userId associated with the active WebSocket session. The server then fetches data only for the authenticated user or their contacts.
- Data Isolation: The getChatHistory method in ChatService ensures that messages are fetched only for conversations between the authenticated user (userId) and a specific contact (friendId).
- No Public Data Exposure: User phone numbers are only visible to contacts that have been added by the user. The getAllUsers endpoint is likely intended for adding new contacts and should be secured in production to prevent enumeration attacks.

Testing

Orbit Chat has been tested to ensure functionality, stability, and a consistent user experience across platforms. The testing process includes unit, integration, and UI testing.

1. Unit Testing

- Component-Level Tests: Individual components (e.g., useSingleChat, useSendChat hooks) are tested for correct state management and data fetching.
- Functionality Tests: Core functions like message sending (sendMessage) and profile image upload (uploadProfileImage) are verified to work as expected.
- State Management: The AuthContext and UserRegistrationProvider are tested to ensure they correctly manage user state and persist data in AsyncStorage.

2. Integration Testing

- Frontend-Backend Communication: The app’s ability to communicate with the Java backend via WebSocket and HTTP requests is thoroughly tested.

    - WebSocket: Messages sent from the frontend are received by the backend, saved to the database, and delivered to the recipient in real-time.
    - HTTP Requests: User registration and profile updates are tested to ensure data is correctly stored in the database.
- Data Consistency: Tests verify that chat history, contact lists, and online status are synchronized between the client and server.

3. UI Testing

- Cross-Platform Compatibility: The app is tested on both Android and iOS devices to ensure consistent layout and functionality.
- Responsive Design: Screens are tested on different screen sizes and orientations to ensure elements resize and reposition correctly.
- Theme Support: Both light and dark modes are tested to ensure all text, backgrounds, and icons are visible and legible.
- Animation & Transitions: Animations (e.g., splash screen, search bar expansion) are tested for smoothness and performance.

4. Load Testing (Future Consideration)

- Scalability: While not currently implemented, load testing would involve simulating multiple concurrent users to assess the system's performance under stress.
- WebSocket Stability: Test how many active WebSocket connections the server can handle before performance degrades.

This comprehensive testing strategy ensures that Orbit Chat provides a reliable and enjoyable user experience.

Deployment

Orbit Chat is deployed as a two-part system: a Java backend server and a React Native mobile application.

1. Backend Deployment

- Server: The Java backend is deployed on a GlassFish or Tomcat application server.
- Database: The MySQL database is hosted on the same server or a separate machine, accessible to the application server.
- Web Server: The server is exposed to the internet using ngrok, which creates a secure tunnel from a public URL (e.g., https://056a5d00dac4.ngrok-free.app) to the local server port. This is primarily used for development and testing.
- Environment Variables: Configuration such as EXPO_PUBLIC_APP_URL and EXPO_PUBLIC_WS_URL are set in the .env file to point to the ngrok URL.

2. Frontend Deployment

- Build Process: The React Native app is built into native packages:
    - Android: An .apk file for installation on Android devices.
- Expo Build: Since you're using Expo, you can use npx expo build:android and npx expo build:ios to generate these files.
- Testing: The app can be tested on physical devices or simulators/emulators before distribution.

Future Enhancements

- Push Notifications: Get alerts for new messages even when the app is closed.
- Voice & Video Calls: Add real-time audio and video chat between users.
- Group Chats: Create and manage conversations with multiple people.
- Media Sharing: Send and view images, videos, and files in chats.
- Message Reactions: React to messages with emojis.
- Message Editing/Deleting: Fix or remove your own messages.

Conclusion

Orbit Chat stands as a fully realized, end-to-end messaging application, showcasing a deep understanding of modern mobile and backend development. From its visually stunning UI to its robust real-time communication engine, the project demonstrates a comprehensive mastery of full-stack development.

Key Achievements

- Complete Feature Set: Successfully implemented core chat functionalities including user authentication, contact management, real-time messaging with delivery/read receipts, and a responsive, theme-aware UI.
- Full-Stack Integration: Seamlessly connected a React Native frontend with a Java EE backend using WebSocket for real-time data and Hibernate for persistent storage.
- Polished User Experience: Delivered a smooth, animated, and intuitive interface that prioritizes user interaction and visual appeal with its distinctive cosmic purple aesthetic.

Lessons Learned

- Real-Time Communication: Gained hands-on experience with javax.websocket and managing persistent connections for live data sync.
- State Management: Mastered using React Context and custom hooks for global state (auth, theme) and dynamic data (chat lists, messages).
- UI/UX Design: Learned to create consistent, animated interfaces using react-native-reanimated and NativeWind.
- Debugging & Problem Solving: Overcame challenges like message rendering, WebSocket connection issues, and avatar URL errors through systematic debugging and code review.

Practical Use

Orbit Chat serves as a functional prototype for a secure, private messaging platform. It's ideal for:

- Learning & Portfolio: Demonstrates advanced skills in React Native, Java, and real-time systems.
- Internal Teams: A private chat app for small groups or companies.
- Foundation for Growth: Ready to be extended into a feature-rich social platform.

Future Potential

The architecture is perfectly positioned for expansion. Future enhancements like push notifications, voice/video calls, group chats, and media sharing are not just ideas—they are clear next steps built on a solid foundation.

Orbit Chat is more than an app; it’s a testament to your technical skill and design vision. It’s ready to evolve, scale, and delight users.

Thank you!